

A 3D rendering of a server rack protected by a glowing shield, with data packets and a Christmas tree in the background.

Comment un éditeur peut-il vendre et maintenir un logiciel métier en mode SaaS (comptes, licences, mises à jour) sans jamais accéder aux données de ses clients — conformément au cadre africain de protection des données (AUDPF) ?

 CHEZ L'ÉDITEUR — NE VOIT JAMAIS LES DONNÉES MÉTIER



Comptes tenants, licences, suivi du parc. Django + PostgreSQL.

voit : compte / licence



Stocke des blobs chiffrés opaques + code de récupération chiffré. Même saisi, il ne déchiffre rien.

voit : jamais le clair

 Frontière cryptographique — tout sort chiffré (DEK · libsodium)



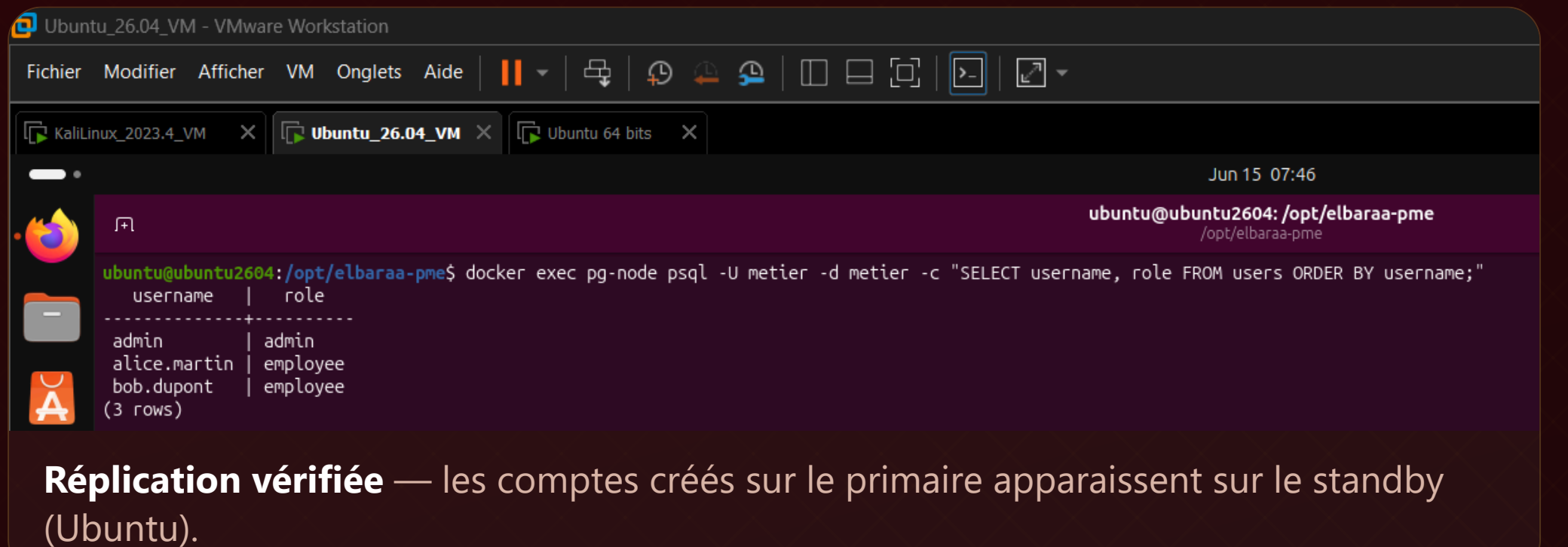
Exécute le logiciel, détient et chiffre les données. PostgreSQL primaire (écritures).

PRIMAIRE · actif

Réplique en continu. Lecture seule. Prend le relais par quorum si le primaire tombe.

STANDBY · réplica

- 🕒 **Réplication streaming PostgreSQL < 1 s** · fencing par jeton d'époque (anti split-brain)



- ✓ Chiffrement **DEK cross-OS** (Windows · Ubuntu · Kali · Debian) — libsodium.
- ✓ Enrôlement d'appareil par **QR code** (sealed box X25519).
- ✓ Récupération **zero-knowledge** (code haute entropie · Argon2id).
- ✓ **Agent IA** (Mistral) supervise les 3 acteurs — sans voir les données.

Rust

lib sodium

XChaCha20-Poly1305

X25519

Argon2id

PostgreSQL

réplication primaire/standby

Docker

CBOR · journal append-only

Django

Agent IA · Mistral

6/6

scénarios métier

4/4

protections
de résilience

2

tenants souverains
indépendants